

Creating and Managing Database Objects

SNOWFLAKE ARCHITECTURE



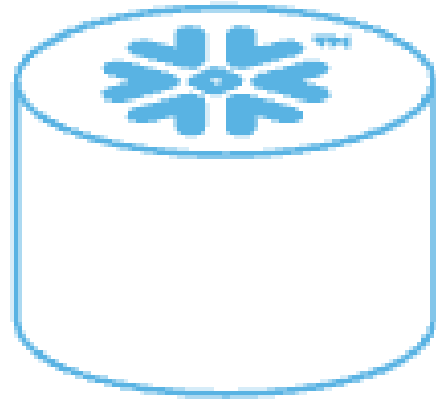
Emily Melhuish

Technical Curriculum Developer,
Snowflake

Creating a Database and Schema

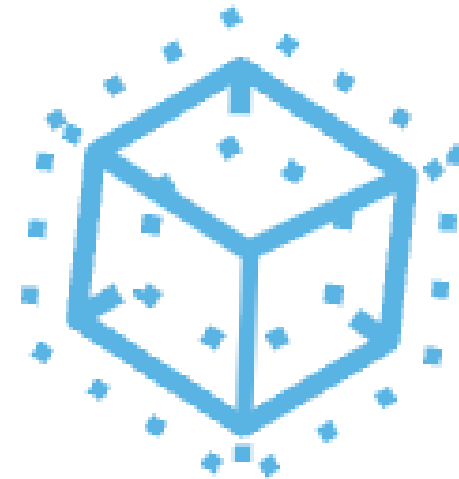
Creating a Database Query

```
CREATE DATABASE snowy_peak_db;
```



Creating a Schema Query

```
CREATE SCHEMA snowy_peak_db.raw;
```



Setting Context

Worksheet – set_context.sql

set_context.sql x Run All

```
1  -- Step 1: set context before running any query
2  USE ROLE ANALYST ;
3  USE WAREHOUSE COMPUTE_WH ;
4  USE DATABASE SNOWY_PEAK_DB ;
5  USE SCHEMA ANALYTICS ;
6
7  -- Step 2: shorthand – fully qualified schema sets db + schema together
8  USE SCHEMA SNOWY_PEAK_DB.ANALYTICS ; -- db.schema in one line
9
10 -- What happens if you skip the warehouse:
11 SELECT * FROM orders LIMIT 10; -- no USE WAREHOUSE before this
12
13
14 -- Fix: set warehouse, then re-run
15 USE WAREHOUSE COMPUTE_WH ;
16
```

role

warehouse

database

schema

SQL execution error: No active warehouse selected for the current session.
Select an active warehouse with the 'use warehouse' command.

All four lines matter
role · warehouse · database
schema – set before running
unambiguous context = no surprises

Shorthand path
USE SCHEMA db.schema
sets both in one line
fully qualified path

No active warehouse error
Most common context mistake
Fix: USE WAREHOUSE, re-run
always the same fix

SNOWFLAKE · WORKSHEETS · CONTEXT SETUP

Querying Your Data

Query

```
SELECT * EXCLUDE(email)
FROM user_registrations
LIMIT 10;
```

Output

Results (just now)

TableChart

10 rows397ms

	USER_ID	FIRST_NAME	LAST_NAME	REGISTRATION_DATE	SUBSCRIPTION_TIER	COUNTRY	REFERRAL_SOURCE	APP_VERSION
	10011010	Camille10.0% Emma10.0% +8 more	Ashworth10.0% Blanchi10.0% +8 more	03/01/202407/02/2024	free40.0% premium40.0% +1 more	GB30.0% FR20.0% +3 more	organic30.0% app_store_search20.0% +3 more	3.2.250.0% 3.2.130.0% +1 more
1	1001	Emma	Thornton	2024-01-03	premium	GB	organic	3.2.1
2	1002	Luca	Ferretti	2024-01-07	free	IT	instagram	3.2.1
3	1003	Sophie	Marchand	2024-01-11	premium	FR	friend_referral	3.2.1
4	1004	James	Okafor	2024-01-15	free	GB	app_store_search	3.2.2
5	1005	Ingrid	Lindqvist	2024-01-19	pro	SE	instagram	3.2.2
6	1006	Marco	Bianchi	2024-01-22	free	IT	organic	3.2.2
7	1007	Hannah	Müller	2024-01-26	premium	DE	friend_referral	3.2.2
8	1008	Tom	Ashworth	2024-01-30	pro	GB	google_ad	3.2.2
9	1009	Camille	Dubois	2024-02-03	free	FR	organic	3.2.3
10	1010	Erik	Svensson	2024-02-07	premium	SE	app_store_search	3.2.3

- Limit does not guarantee a cheaper query

Understanding Your Tables

Query

```
SELECT table_name, row_count, bytes
FROM information_schema.tables
WHERE table_schema = 'PRODUCT';
```

Output

Results (just now) ⌵			
Table Chart		🔍 📄 2 rows ⓘ 1.7s ⬇ ⌚	
🔍	<u>A</u> TABLE_NAME	# ROW_COUNT	# BYTES
1	IN_APP_PURCHASES	10	4096
2	APP_SESSIONS	10	4096

Understanding Your Columns

Query

```
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'IN_APP_PURCHASES';
```

Output

Results (just now)

Table

Chart

10 rows

1.7s

	A COLUMN_NAME	A DATA_TYPE	A IS_NULLABLE
	AMOUNT_USD10.0%	TEXT80.0%	
	CURRENCY10.0%	DATE10.0%	
	+8 more	+1 more	YES100.0%
1	PLATFORM	TEXT	YES
2	USER_ID	TEXT	YES
3	REGION	TEXT	YES
4	ITEM_TYPE	TEXT	YES
5	AMOUNT_USD	NUMBER	YES
6	PURCHASE_ID	TEXT	YES
7	STATUS	TEXT	YES
8	ITEM_NAME	TEXT	YES
9	PURCHASE_DATE	DATE	YES
10	CURRENCY	TEXT	YES

RBAC: Role-Based Access Control

Common Privileges

- `SELECT` to read data
- `INSERT` , `UPDATE` and `DELETE` to write data
- `USAGE` to access a database, schema, or warehouse
- `OWNERSHIP` for full structural control

RBAC: Granting Privileges in Practice

Granting access to the analyst role

```
GRANT USAGE ON DATABASE snowy_peak_db TO ROLE analyst;  
GRANT USAGE ON SCHEMA product TO ROLE analyst;  
GRANT SELECT ON TABLE product.in_app_purchases TO ROLE analyst;
```

- Users do not get privileges directly
 - They are assigned roles
- Principle of least privilege

Let's practice!
SNOWFLAKE ARCHITECTURE

Virtual Warehouses: Types, Sizing, and Scaling

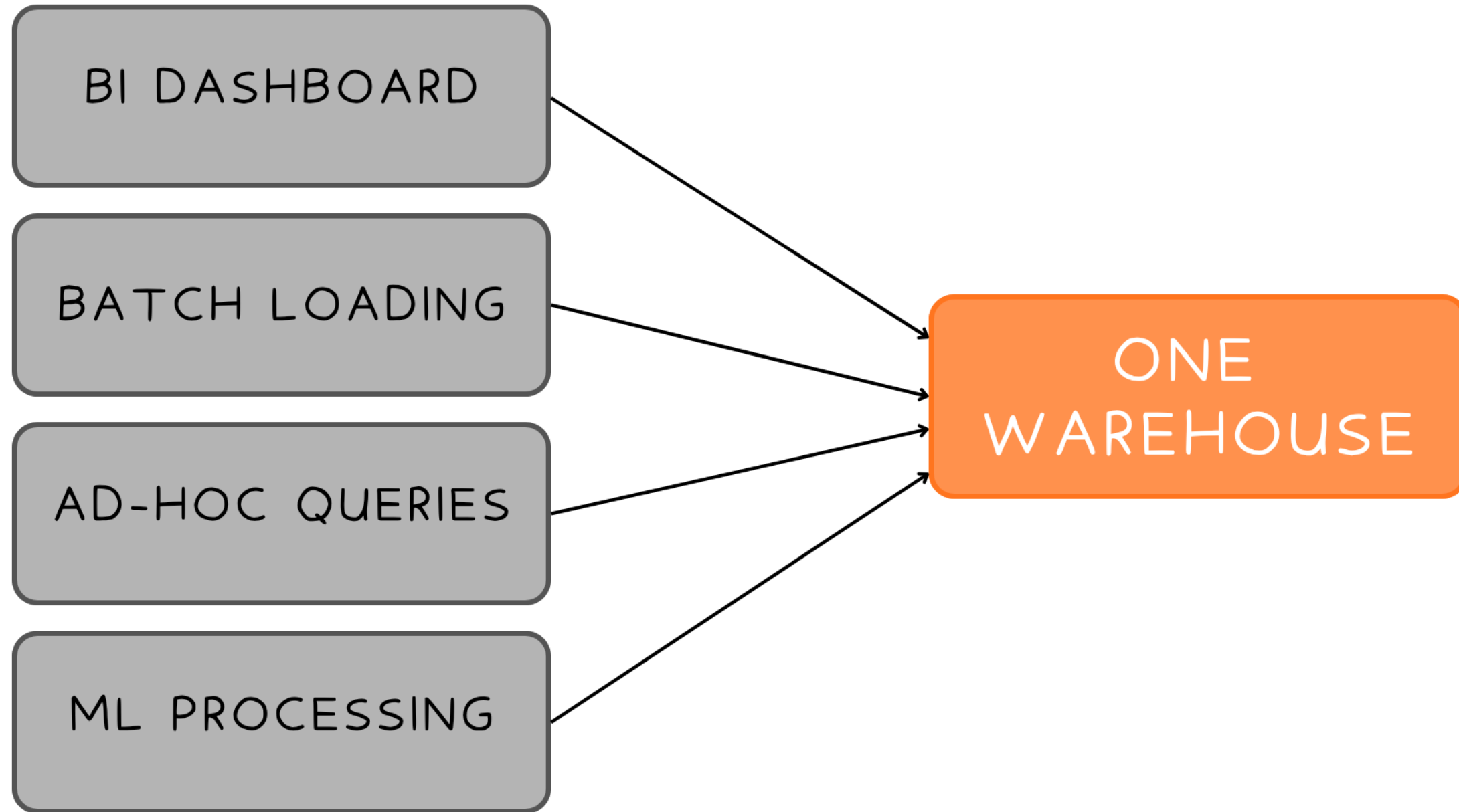
SNOWFLAKE ARCHITECTURE



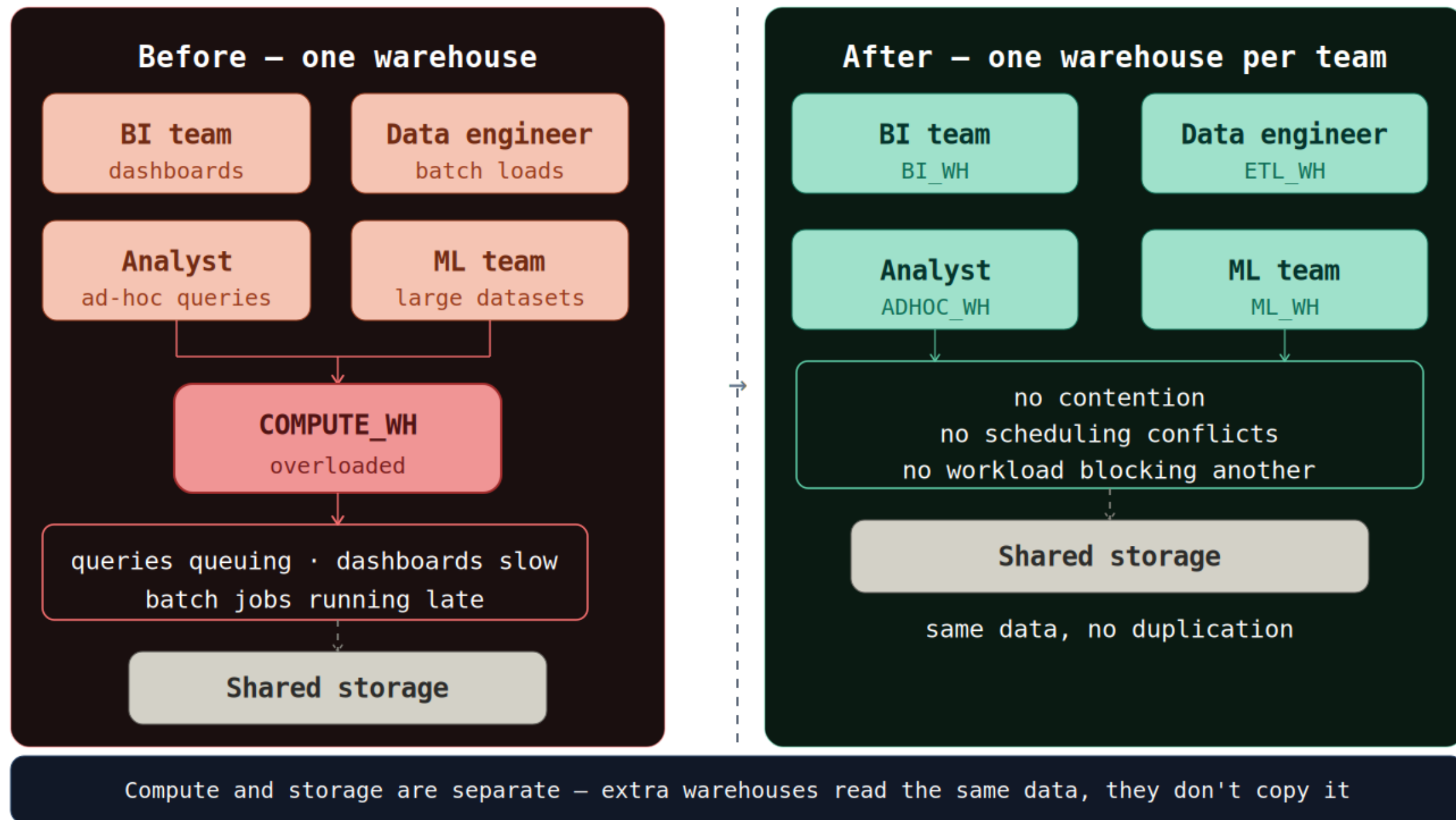
Emily Melhuish

Technical Curriculum Developer,
Snowflake

One Warehouse, Many Workloads



One Warehouse, Too Many Workloads



Standard vs Snowpark Optimized

Standard:

- Use cases:
 - SQL queries
 - Data loading
 - BI reporting
- Gen2 option available: Improved performance
- Test Gen1 vs Gen2 for your workload if cost matters

Snowpark Optimized

- Use cases:
 - ML model training
 - Snowpark DataFrames
 - Memory-intensive
- Available from Medium upward

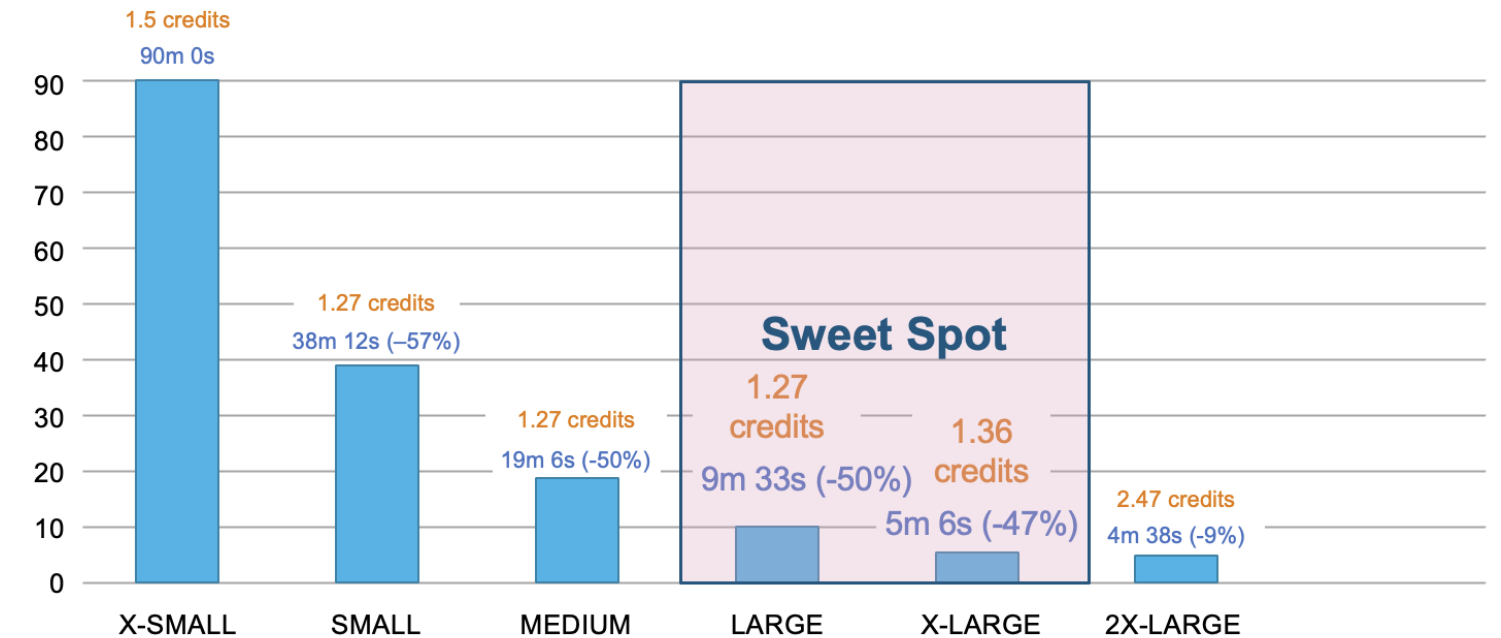
Warehouse Sizing

Warehouse size use cases:

- XS = Light queries
- XL = Large and Complex work

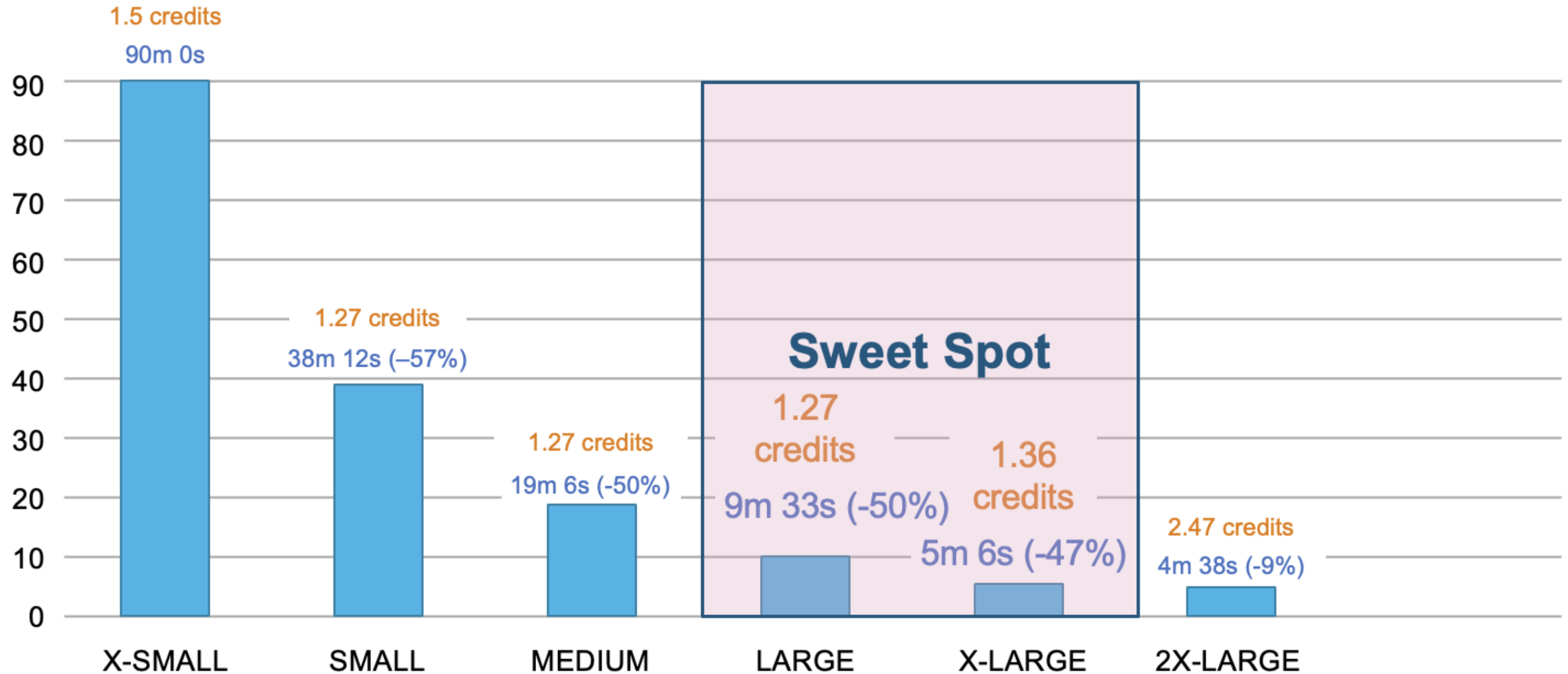
Query

```
CREATE WAREHOUSE analytics_wh  
  WAREHOUSE_SIZE = 'LARGE'  
  AUTO_SUSPEND = 300  
  AUTO_RESUME = TRUE;
```



- bills per second with a 60-second minimum

The Sweet Spot



Cost Control: Auto-Suspend and Auto-Resume

Cost Control Setting Method 1

Auto-Suspend

- Defines seconds of inactivity before warehouse suspends
- Automatic suspension
- 300 seconds (5m) = DEFAULT

Cost Control Setting Method 2

Auto-Resume

- = TRUE means it wakes up when a query arrives
- No manual intervention required

Multi-cluster Warehouse

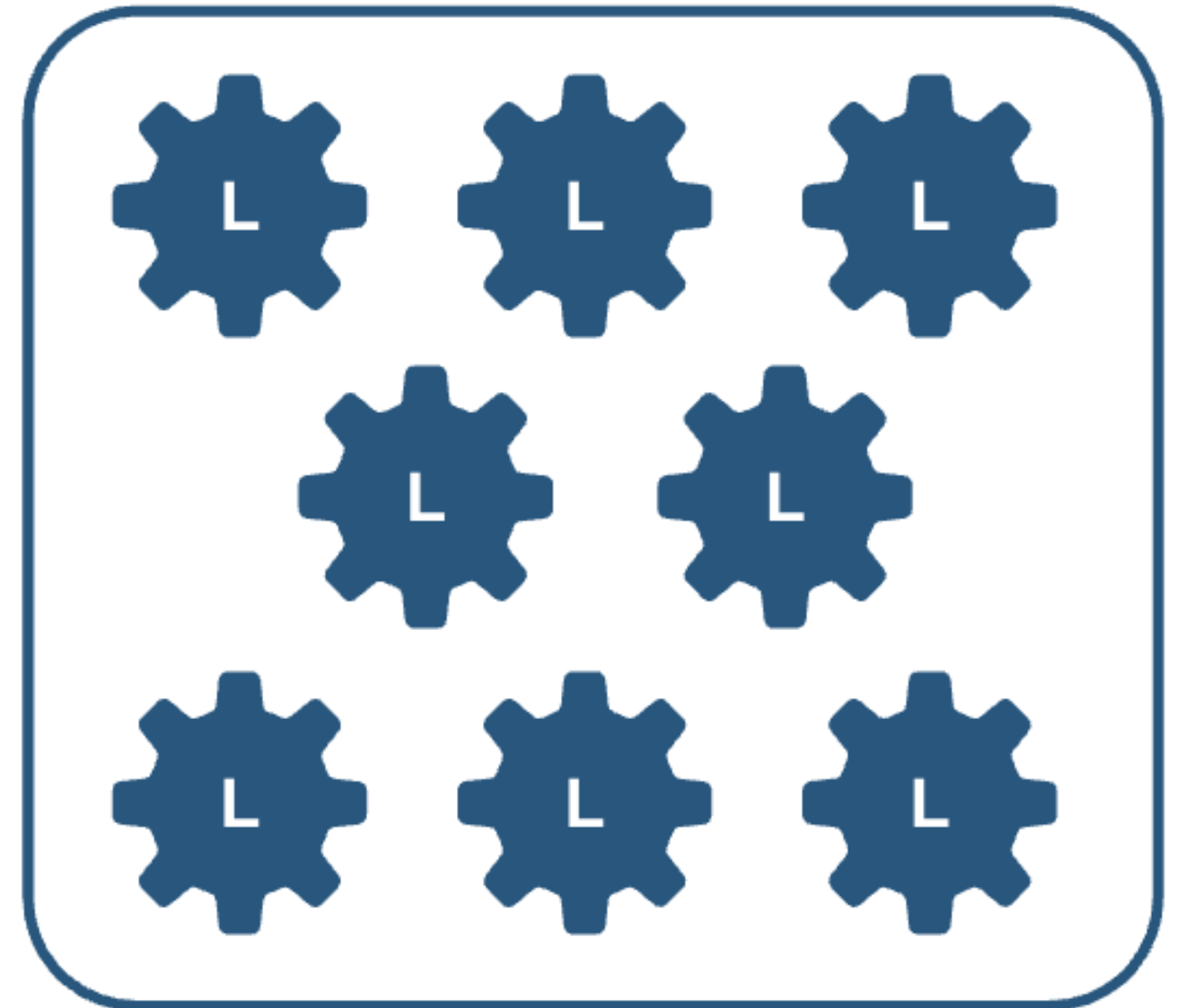
Use Case:

- Answer to high-concurrency workloads
- Clusters spin up/suspend automatically

Query

```
CREATE WAREHOUSE reporting_wh  
  WAREHOUSE_SIZE = 'LARGE'  
  MIN_CLUSTER_COUNT = 1  
  MAX_CLUSTER_COUNT = 3  
  SCALING_POLICY = 'STANDARD';
```

- Enterprise edition feature



Scaling Up vs Scaling Out

CREDITS PER HOUR

NUMBER OF CLUSTERS

VIRTUAL WAREHOUSE SIZE

	1	2	3	4	5	6	7	8	9	10
4XL	128	256	384	512	640	768	896	1024	1152	1280
3XL	64	128	192	256	320	384	448	512	576	640
2XL	32	64	96	128	160	192	224	256	288	320
XL	16	32	48	64	80	96	112	128	144	160
L	8	16	24	32	40	48	56	64	72	80
M	4	8	12	16	20	24	28	32	36	40
S	2	4	6	8	10	12	14	16	18	20
XS	1	2	3	4	5	6	7	8	9	10

XL x1 = 16 credits per hour
Scale up for complexity

M x4 = 16 credits per hour
Scale out for concurrency

Economy vs Standard Scaling Policy

Standard

- Spins up when anything is queued
- Throughput over savings
- Unpredictable Workloads

Economy

- Conservative
- Load will keep it busy >6 minutes
- Steady and predictable workloads

Let's practice!
SNOWFLAKE ARCHITECTURE

Loading Structured and Semi-Structured Data

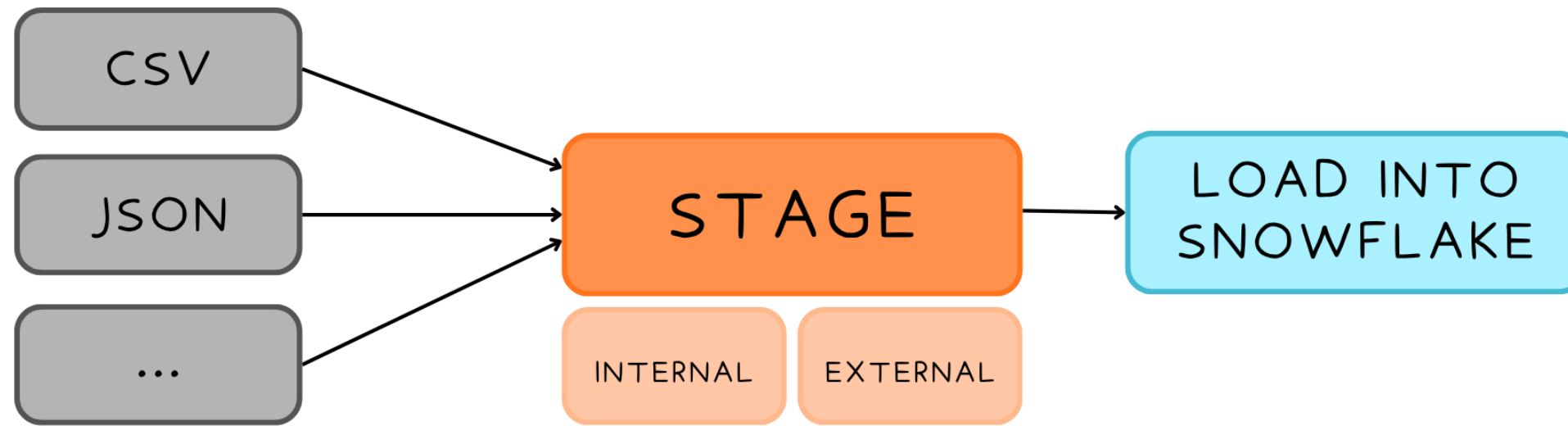
SNOWFLAKE ARCHITECTURE



Emily Melhuish

Technical Curriculum Developer,
Snowflake

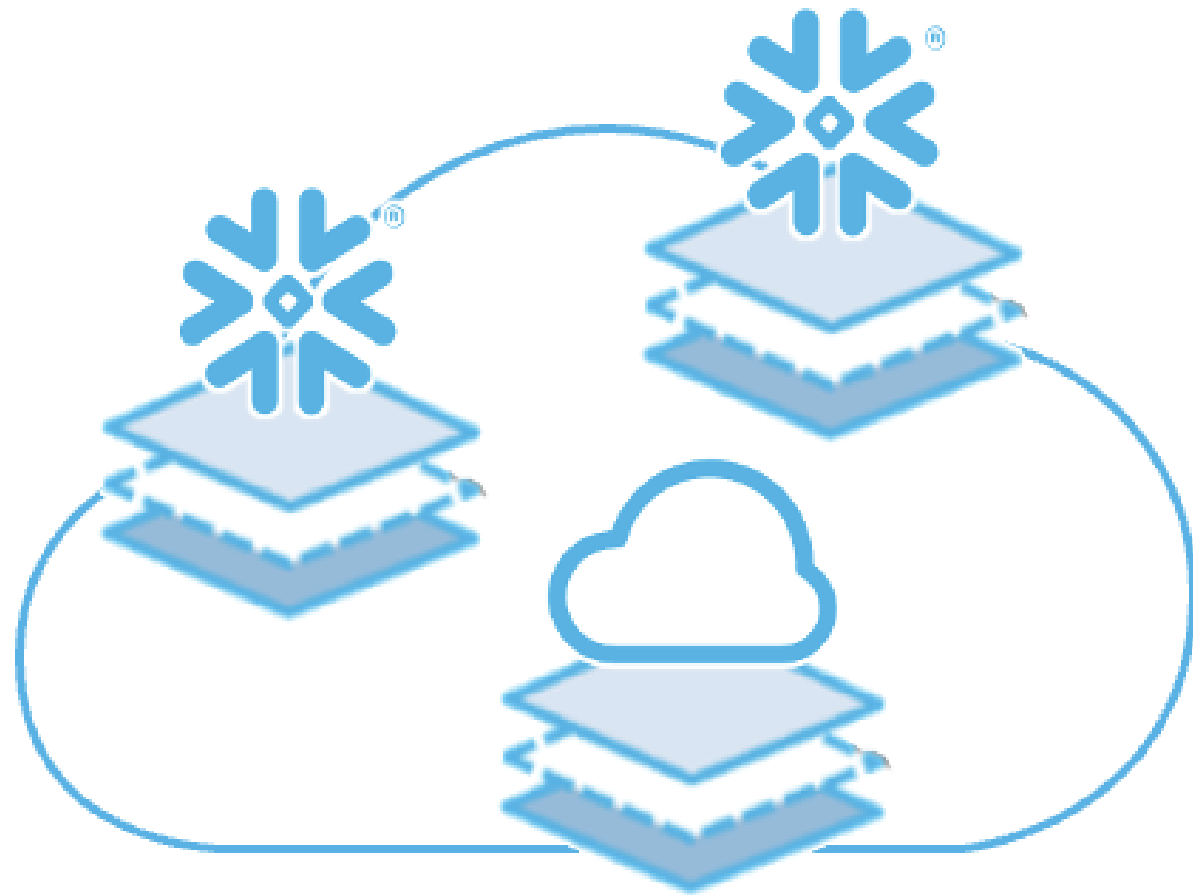
The Loading Workflow



- Files land in the stage, execute `COPY INTO` to load them

What is a Stage?

Stage = temporary storage location



Create Internal Stage

```
CREATE STAGE snowy_peak_stage;
```

Create External Stage

```
CREATE STAGE my_s3_stage  
  STORAGE_INTEGRATION = s3_int  
  URL = 's3://snowy-peak-data/raw/';
```


Loading data into Snowflake

COPY INTO

- Bulk loading from Stage
- Structured and Semi-Structured Data
- Scalable

INSERT

- Great for a small number of rows without staging
- One-off additions or minor changes

Snowsight UI

- Non-technical users and quick one-off workloads

Inspecting a Stage with LIST

Worksheet - inspect_stage.sql

inspect_stage.sql Run

```
1  -- inspect the stage before loading
2  LIST @ snowy_peak_stage ;
```

RESULTS 4 files

NAME	SIZE	LAST MODIFIED
snowy_peak_stage/orders/2024-03-01.csv.gz	1.2 MB	2024-03-01 08:14:02
snowy_peak_stage/orders/2024-03-02.csv.gz	1.4 MB	2024-03-02 08:09:55
snowy_peak_stage/orders/2024-03-03.csv.gz	1.1 MB	2024-03-03 08:22:10
snowy_peak_stage/orders/2024-03-04.csv.gz	1.3 MB	2024-03-04 08:05:43

```
9
10 -- files confirmed -- ready to load
11 COPY INTO orders FROM @ snowy_peak_stage /orders/ -- next step
12
13
14
15
16
```

@ means stage
@ before a name tells Snowflake it's a stage, not a table

What LIST returns
name · size · last modified
every file in the stage,
no arguments needed

Pre-load check
Confirm right files exist
before committing to
COPY INTO

SNOWFLAKE · STAGES · LIST COMMAND

Loading Structured Data with COPY INTO

Query

```
CREATE OR REPLACE FILE FORMAT csv_format
  TYPE = CSV
  SKIP_HEADER = 1;

COPY INTO orders
FROM @snowy_peak_stage/orders.csv
FILE_FORMAT = (FORMAT_NAME = csv_format);
```

Loading and Querying Semi-Structured Data

Loading

JSON loads into a special Snowflake column type called VARIANT.

```
CREATE TABLE events (raw_event VARIANT);

COPY INTO events
FROM @snowy_peak_stage/events.json
FILE_FORMAT = (TYPE = JSON);
```

Querying

Use colon notation to navigate the JSON structure,

```
SELECT
  raw_event:user_id::STRING AS user_id,
  raw_event:event_type::STRING
  AS event_type
FROM events;
```

Loading with the UI

The screenshot displays the Snowflake 'Add data' interface. On the left is a sidebar with navigation options: 'Work with data' (containing 'Projects', 'Ingestion' (highlighted), 'Transformation', 'AI & ML', 'Monitoring', and 'Marketplace'), 'Horizon Catalog' (containing 'Catalog', 'Data sharing', and 'Governance & security'), and 'Manage' (containing 'Compute' and 'Admin'). The main area is titled 'Add data' with the subtitle 'Bring your data into Snowflake'. It features a grid of 15 options:

- Load data into a Table (cloud upload icon)
- Load files into a Stage (folder icon)
- Snowflake Stage (Snowflake logo icon)
- AWS S3 (AWS logo icon)
- Microsoft Azure Blob Storage (Azure logo icon)
- Google Cloud Storage (Google Cloud logo icon)
- Learn how to get data from a URL (link icon, with an external link symbol)
- Learn about Snowpipe Streaming (pipe icon, with an external link symbol)
- Snowflake Marketplace (marketplace icon)
- Snowflake Connector for ServiceNow (Snowflake logo icon)
- Snowflake Connector for Google Analytics Aggregate Data (Snowflake logo icon)
- Snowflake Connector for Google Analytics Raw Data (Snowflake logo icon)
- Snowflake Connector for MySQL (Snowflake logo icon)
- Snowflake Connector for PostgreSQL (Snowflake logo icon)
- SNP Glue Connector for SAP (SNP logo icon)

A [More connectors](#) link is located at the bottom left of the main area.

Let's practice!
SNOWFLAKE ARCHITECTURE